Model-Driven Software Development

External DSL background

Miguel Campusano

SDU Software Engineering

Previous class

Internal DSL

Summary

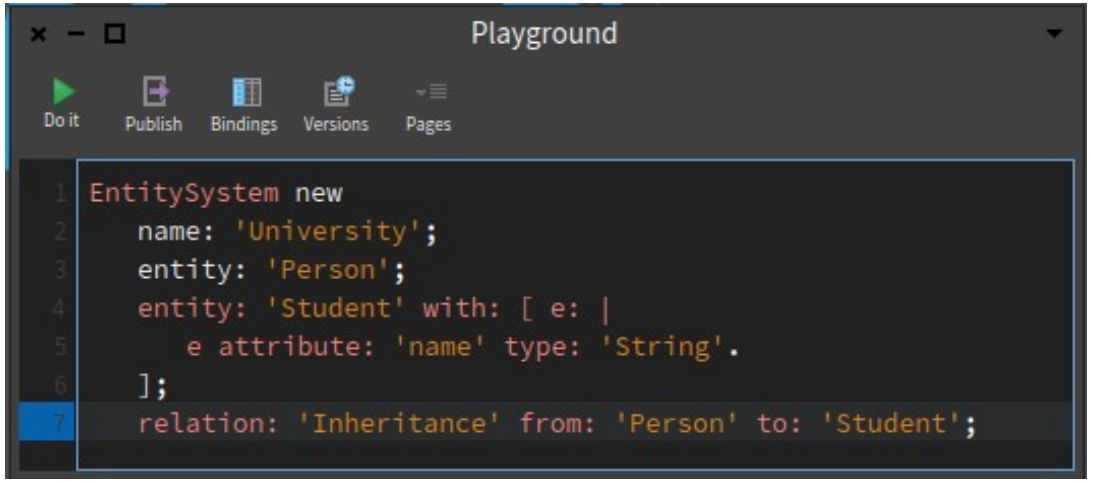
Warm-up Internal DSL

Internal DSL Syntax

Internal DSL Patterns

```
public void build() {
    system("University").
        entity("Person").
            attribute("name", "String").
            attribute("age", "Number").
        entity("Student").
            attribute("id", "Number").
        entity("Teacher").
        relation("Inheritance", "Person", "Student").
        relation("Inheritance", "Person", "Teacher");
}
```

```
#lang racket
(entitysystem "University"
              (entity "Person"
                      (attribute "name" "String")
                      (attribute "age" "Number"))
              (entity "Student"
                      (attribute "id" "Number"))
              (entity "Teacher")
              (relation "Inheritance" "Person" "Student"))
```



Internal DSL Patterns

Metamodel (Semantic Model)

Expression builder (fluent-API)

Context Variable (what we operate on)

Construction Builder (immutable objects)

Symbol table (names)

Class symbol table (static typing)

Class diagram metamodel / model

M2: Metamodel

describes

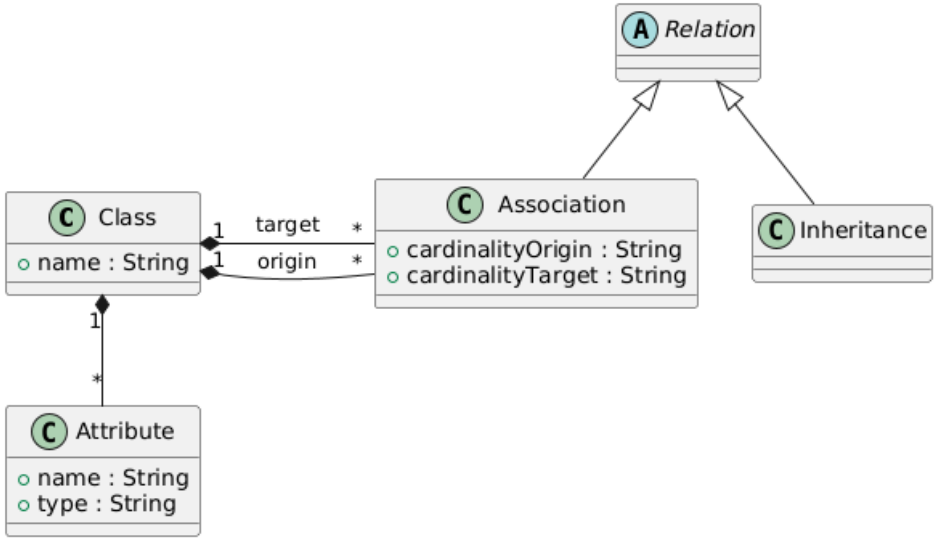
instance of

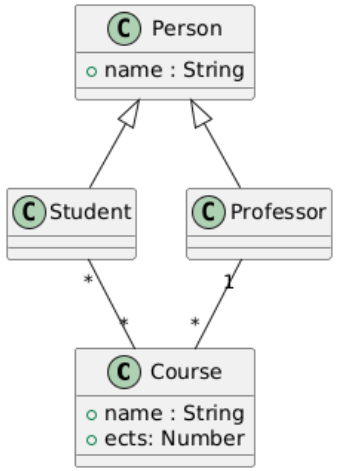
M1: Model

describes

instance of

M0: Heap





Objects in memory
Class: Person
Class: Student
Class: Professor
...

Discussion

Internal DSL

- Advantages of internal DSLs
  - Quick to implement, natural evolution
  - No additional tools, languages, infrastructure
- Disadvantages of internal DSLs
  - Model-based API can be cumbersome and confusing
  - Lack of static checking, IDE support (*cleverness* can not solve everything)
  - Mix of host language and DSL enables abuse

External DSL

- Compiler that translates text into a model
- Feels more like a *real language*
  - Dedicated grammar (IDE, language, syntax, …)
  - Natural fit for code generation
- Requires construction of a *real language*
  - Significant work to build compiler infrastructure
  - Even more work to provide proper IDE support
    - Complex problem domain

Discussion: Internal DSL vs External DSL

We have discussed about internal DSLs

- Advantages / Disadvantages

We have discussed *some* on external DSLs

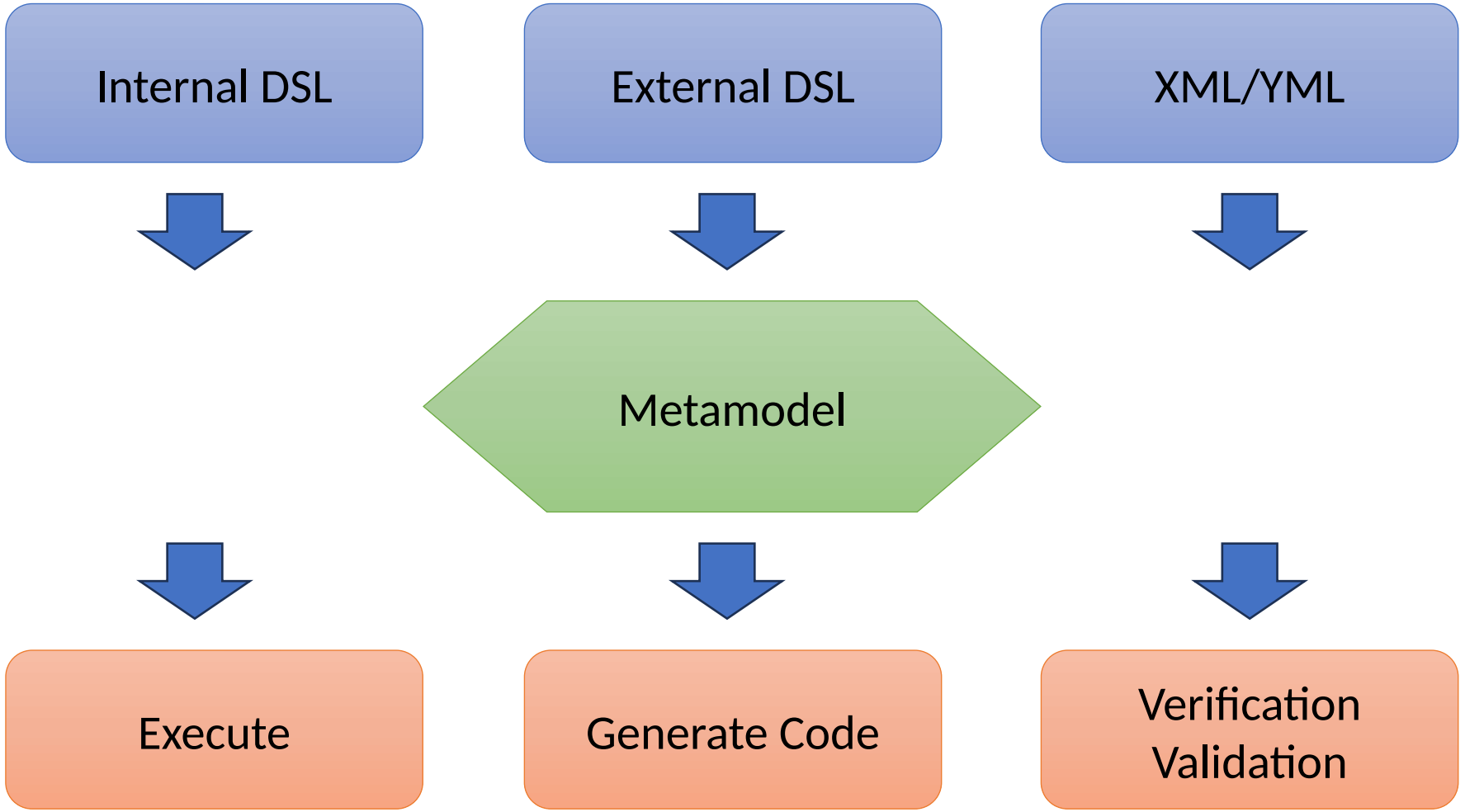
- Advantages / Disadvantages

You have to develop both for your projects.

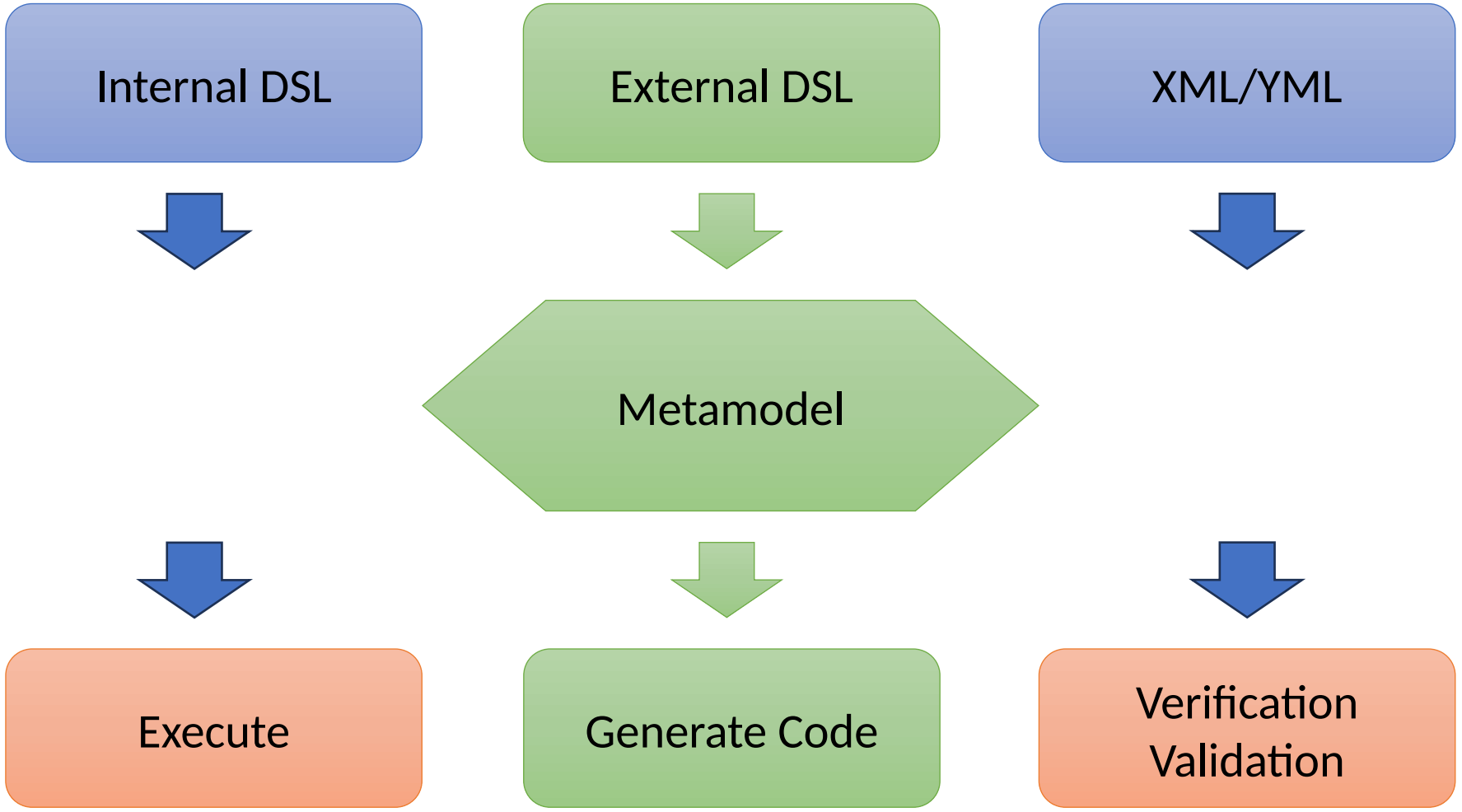
However, if you have the chance to pick only one option...

- which one would you choose, and why?

Metamodel


Internal DSL
External DSL
XML/YML
Metamodel
Execute
Generate Code
Verification
Validation

Metamodel: External DSL & Code Generation



Model-Driven Software Development

External DSL background

Miguel Campusano

SDU Software Engineering

Today

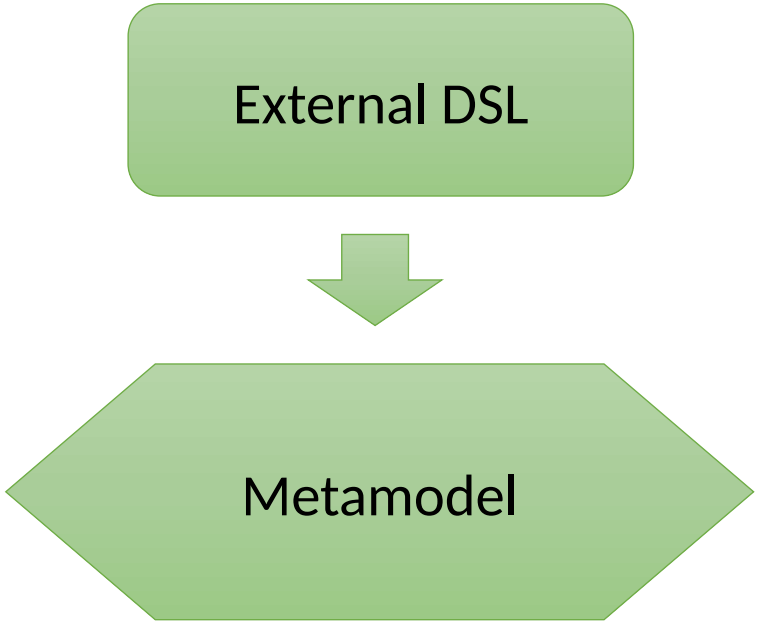
Class Diagram example syntax definition

From External DSL to Metamodel

From *Program* to Metamodel

From *Text* to Metamodel

XText warm-up

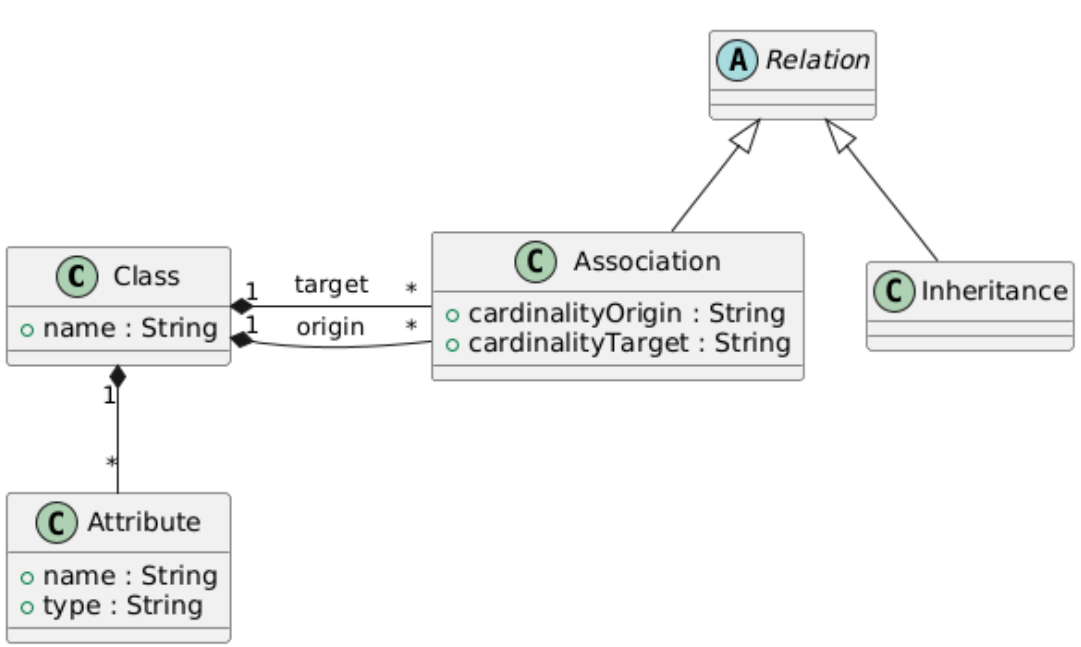


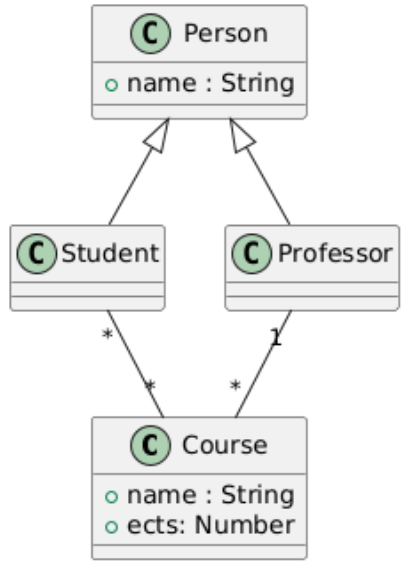
Class Diagram syntax

Metamodel – External DSL

M2: Metamodel —describes→ M1: Model

M1: Model ←instance of— M2: Metamodel





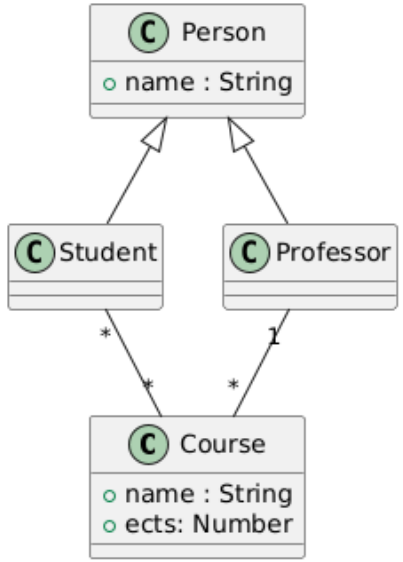
Class Description example syntax

```
system("University")
  .entity("Person")
    .attribute("name", "String")
    .attribute("age", "Number")
  .entity("Student")
    .attribute("id", "Number")
  .entity("Teacher")
  .entity("Course")
    .attribute("title", "String")
  .relation("Inheritance", "Person", "Student")
  .relation("Inheritance", "Person", "Teacher")
  .relation("1-1", "Course", "Teacher")
    .relation("Many-Many", "Course", "Student") ;
```

Text to Metamodel

Text to Metamodel instance

Class Diagram Textual Representation to Metamodel instance



Text to Metamodel instance

- Lexing
  - String to Tokens (Strings with meaning)
- Parsing
  - Tokens to Data Structure (Usually an Abstract Syntax Tree)
- Name resolution
  - Identifiers associated to other elements
    - Function Call *resolves* to a Function Definition
    - Variable Usage *resolves* to Variable Definition
- ...

Context-Free Grammar

Before: regular expressions

Regular Expressions for balanced parentheses, match:

```
( )
( )( )
(( ))
(( ) ( ) (( )))
...
```

PollEv.com/mica

Context-Free Grammar

Formal grammar specified by several production rules

- Rewrite rule; substitute left side for right side

`A -> α`

- Single nonterminal symbol at the left (A)
- String of terminals and/or non-terminals at the right (α)

Describe context-free languages

- Describe programming languages syntax

Context-Free Grammar: BNF

Backus-**N**aur **F**orm

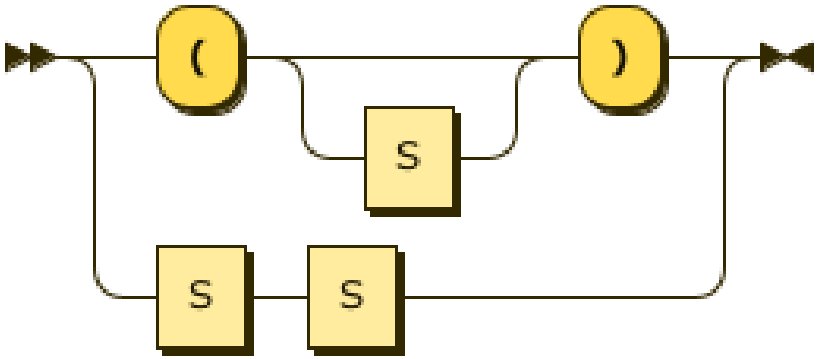
Notation to define context-free grammars

DSL for specifying programming languages

BNF for balanced parentheses?

BNF for balanced parentheses

```
S ::= ‘(‘ ‘)’
S :: = ‘(‘ S ‘)’
S ::= SS
```



Source: https://www.bottlecaps.de/rr/ui

In-class exercise: BNF

While loop

IDs => `aVariable - ClassName - Report20 - Report20a`

Numbers => `1 - 20 - 0.3 – 0.04 - 0.056`

Consider the following rules:

```
Letter ::= a | b | … | z | A | B |…| Z ;
Digit ::= 0 | 1 | 2 | … | 9 ;
```

Context-Free Grammar: EBNF

Extended BNF

- BNF with quality-of-life symbols
- It is not more *powerful*
  - Anything you express in EBNF can be expressed using BNF
  - It is more convenient

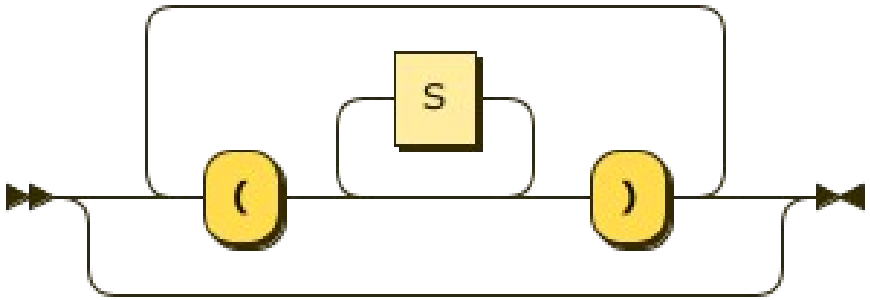
Extensions

- * Kleene star: 0 or more occurrences
- + Kleene plus: 1 or more occurrences
- ? Optional: 0 or 1 occurrences

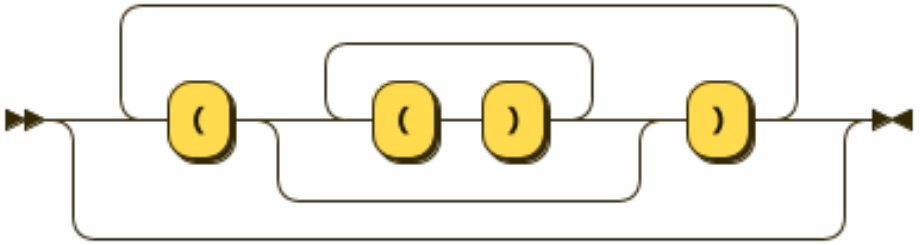
EBNF for balanced parenthesis?

EBNF for balanced parentheses

```
S ::= ( ‘(‘ S* ‘)’ )*
```



Source: https://www.bottlecaps.de/rr/ui



In-class exercise: EBNF

EBNF for IDs:

```
  aVariable
  ClassName
  Report20
  Report20a
…
```

EBNF for numbers:

```
  1
  20
  0.3
  0.04
  0.056
…
```

Consider the following rules:

```
Letter ::= a | b | … | z | A | B |…| Z ;
Digit ::= 0 | 1 | 2 | … | 9 ;
```

In-class exercise: While loop

EBNF for while loop:

```
while (<expression>) {
  <statement>
  <statement>
  …
  <statement>
}
```

Context-Free Grammars

Describe the structure of sentences

Describe context-free languages

In our course, define the structure of programming languages

Syntax

Same *languages* can be described with different grammars (EBNF)

```
S ::= ( ‘(‘ S* ‘)’ )*
S ::= ( ‘(‘ S* ‘)’ )* ( ‘(‘ S* ‘)’ )*
```

No context: there are *properties* you cannot write

For example: Cycle inheritance

We will see more on future classes

Class Diagram grammar

In-class exercise: Class Description EBNF

```
system University
 entity Person {
    attribute name as String
    attribute age as Number
 }
 entity Student {
    attribute id as Number
 }
 entity Teacher {
 }
 entity Course {
    attribute title as String
 }
 relations {
    Student inherits Person
    Teacher is Person
    Course *-* Student
 }
```

XText

Context-Free Grammar: Xtext

Not quite EBNF, but more

- EBNF + Metamodel definition

Basically:

- EBNF allows for parsing and lexing
- Xtext EBNF also allows for name resolution, then, *automatically*:
  - Metamodel definition through EBNF
  - Metamodel instance from a DSL program

Text to Metamodel instance

- Lexing
  - String to Tokens (Strings with meaning)
- Parsing
  - Tokens to Data Structure (Usually an Abstract Syntax Tree)
- Name resolution
  - Identifiers associated to other elements
    - For example: Function Call *resolves* to a Function Definition

Text to Metamodel instance

- Lexing
  - String to Tokens (Strings with meaning)
- Parsing
  - Tokens to Data Structure (Usually an Abstract Syntax Tree)
- Name resolution
  - Identifiers associated to other elements
    - For example: Function Call *resolves* to a Function Definition

Summary

Summary

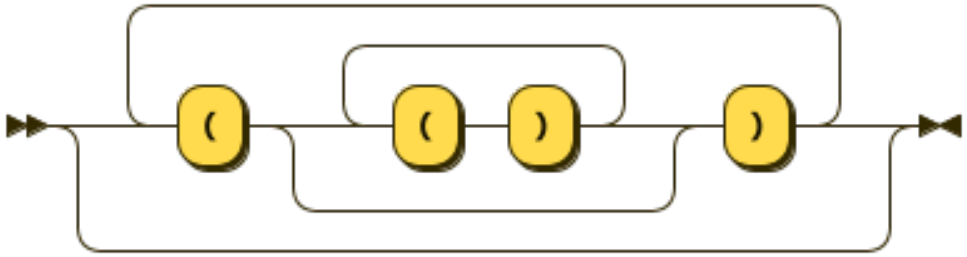
Entity example syntax definition

From External DSL to Metamodel

From *Program* to Metamodel

From *Text* to Metamodel

XText warm-up

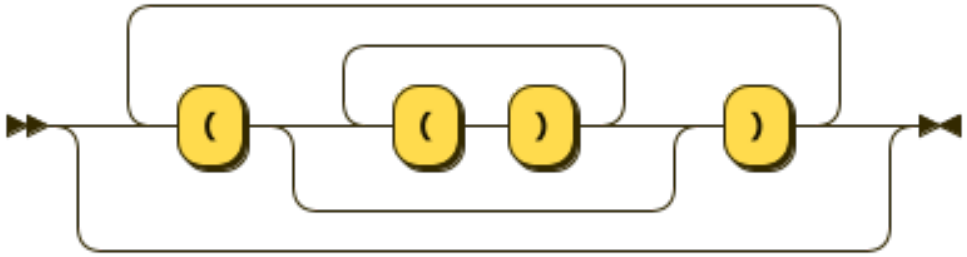


Xtext

Group Project Work

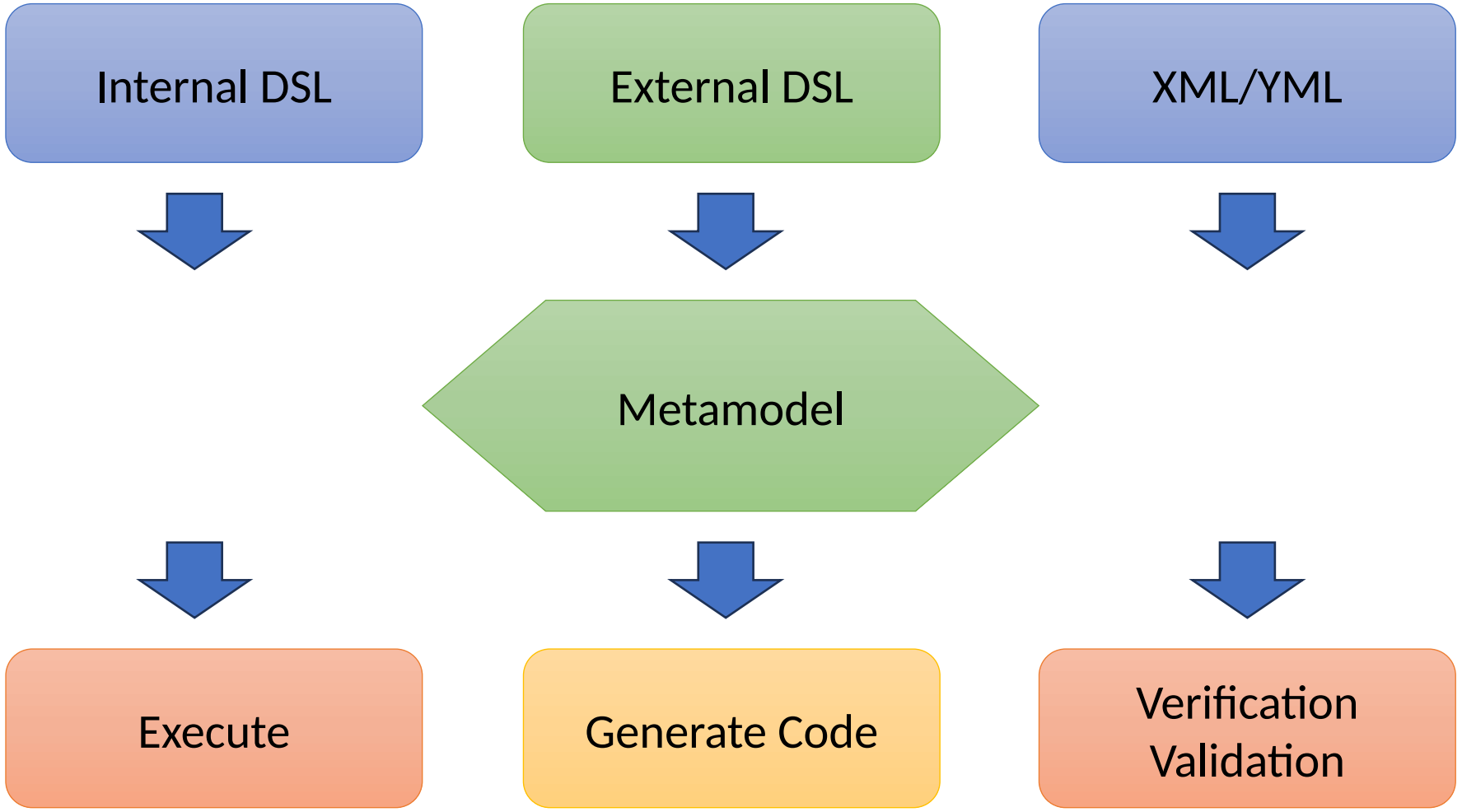
Week 4

- EBNF specification
- Xtext Grammar



Xtext

Next


Internal DSL
External DSL
XML/YML
Metamodel
Execute
Generate Code
Verification
Validation